Lecture L08

*Blind review — please score this lecture on row **L08** in the Excel form (1–10 per dimension).*

Learning Objectives

By the end of this lecture, you will be able to: 1. **Explain** the intuition of backpropagation as a recursive application of the chain rule. 2. **Calculate** the gradient of a cost function with respect to weights in a simple network using a step-by-step derivation. 3. **Analyze** how the vanishing gradient problem arises from the mathematical structure of the chain rule. 4. **Compare** the efficiency of backpropagation (dynamic programming) against naive gradient computation.

---

1. The Core Paradox: The Credit Assignment Problem

Imagine you are managing a massive organization with 100 layers of hierarchy. A final product is delivered to a customer, and it is a disaster. You know the final result is wrong, but you have 10 million employees (weights) across those 100 layers.

The Paradox: How do you know exactly which employee in the 12th layer made the specific mistake that caused the failure at the 100th layer? If you punish everyone equally, you destroy the parts of the system that actually worked. If you only punish the final inspector, the root cause remains.

This is the **Credit Assignment Problem**. Backpropagation is the mathematical solution to this problem: it is a precise accounting system that distributes "blame" for the final error back through the network to every single parameter.

---

2. Prerequisites and Conceptual Framework

2.1 Forward Propagation

In a feedforward network, data flows in a single direction. An input vector $\mathbf{x}$ passes through successive layers of weights $\mathbf{W}$ and biases $\mathbf{b}$, processed by an activation function σ, to produce a prediction $\hat{y}$.

The computational sequence is:

$$\text{Input} \to \text{Hidden Layers} \to \text{Output } (\hat{y}) \to \text{Cost } J(\theta)$$

2.2 The Cost Function $J(\theta)$

The cost function $J(\theta)$ quantifies the error between the prediction $\hat{y}$ and the target value y. The objective of training is to minimize J by adjusting the parameters θ.

2.3 The Chain Rule: The "Gear Train" of Calculus

Backpropagation is the application of the chain rule to a composition of functions. If a variable y depends on u, and u depends on x, the derivative of y with respect to x is:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial x}$$

Technical Analogy: Think of the chain rule as a series of connected gears. If gear A turns gear B, and gear B turns gear C, the ratio of how much gear C moves relative to gear A is the product of the ratios of each individual connection. If one gear in the middle is jammed (derivative = 0), no matter how fast you turn gear A, gear C will not move.

Quick Check 1: Given $f(x) = \sin(x^2)$, identify the two links in the chain required to find the derivative with respect to x. *Answer: Let* $u = x^2$. *Then* $\frac{df}{du} = \cos(u)$ *and* $\frac{du}{dx} = 2x$. *The result is* $2x\cos(x^2)$.

---

3. Intuition: The Logic of Backpropagation

The "Blame Game" Logic

Consider a factory assembly line where the final product is defective, resulting in a high cost J. 1. The manager asks the final inspector (Output Layer): "How much did your last adjustment contribute to this error?" 2. The inspector responds: "I only followed the instructions from the assembly worker (Hidden Layer)." 3. The manager then asks the assembly worker: "How much did your settings contribute to the inspector's error?"

This process moves backward from the error to the source, assigning "blame"—represented mathematically as gradients—to each parameter.

Formalizing the Intuition

For a sequence of computations $z \to a \to \hat{y} \to J$, the effect of z on J is computed by multiplying the local derivatives along the path:

$$\frac{\partial J}{\partial z} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a} \cdot \frac{\partial a}{\partial z}$$

Technical Distinction: Backpropagation does not update weights. It exclusively computes the gradients. A separate optimization algorithm, such as Stochastic Gradient Descent (SGD), uses these gradients to perform the actual weight update.

---

4. The Mathematical Mechanism for Multilayer Perceptrons (MLP)

We define the operations for a layer l as follows: - Weighted input: $z^{(l)} = w^{(l)}a^{(l-1)} + b^{(l)}$ - Activation: $a^{(l)} = \sigma(z^{(l)})$

Step A: The Output Error ($\delta^{(L)}$)

We define the error of the output layer L as $\delta^{(L)}$, representing the gradient of the cost with respect to the weighted input:

$$\delta^{(L)} = \frac{\partial J}{\partial z^{(L)}} = \frac{\partial J}{\partial a^{(L)}} \odot \sigma'(z^{(L)})$$

The error depends on the sensitivity of the cost to the output, scaled by the slope of the activation function.

Step B: Error Propagation (The Recursive Step)

The Tension: If we had to calculate the gradient for every weight from scratch, we would repeat the same calculations millions of times. In a network with 100 layers, calculating the gradient for the first

layer would require re-traversing the entire network 100 times. How do we avoid this computational collapse?

We use the result from the layer we just finished. Having computed $\delta^{(L)}$, we propagate the error backward to layer l using the error from layer $l+1$:

$$\delta^{(l)} = ((\mathbf{w}^{(l+1)})^T \delta^{(l+1)}) \odot \sigma'(z^{(l)})$$

The error at the current layer is the weighted sum of the errors in the subsequent layer, filtered through the current layer's activation slope.

Step C: Gradient Computation

With $\delta^{(l)}$ defined, the gradients for the weights and biases are:

$$\frac{\partial J}{\partial w^{(l)}} = \delta^{(l)}(a^{(l-1)})^T, \quad \frac{\partial J}{\partial b^{(l)}} = \delta^{(l)}$$

---

5. Fully Worked Example: Single Path

Scenario: - Input: $x = 1.0$, Weight: $w = 0.5$, Activation: $\sigma(z) = \text{sigmoid}(z)$, Target: $y = 1.0$, Cost: $J = \frac{1}{2}(y - \hat{y})^2$

Forward Pass: 1. $z = w \cdot x = 0.5 \cdot 1.0 = 0.5$ 2. $\hat{y} = \sigma(0.5) \approx 0.622$

Backward Pass: 1. Compute $\frac{\partial J}{\partial \hat{y}}$: $-(y - \hat{y}) = -(1.0 - 0.622) = -0.378$ 2. Compute $\frac{\partial \hat{y}}{\partial z}$: $\sigma'(0.5) = \hat{y}(1 - \hat{y}) = 0.622(0.378) \approx 0.235$ 3. Compute $\frac{\partial z}{\partial w}$: $x = 1.0$ 4. Apply the chain rule:

$$\frac{\partial J}{\partial w} = (-0.378) \cdot (0.235) \cdot (1.0) = -0.0888$$

Try This Yourself: If the target y was 0.622 (exactly our prediction), what would the gradient be? Why does this make intuitive sense for a system trying to minimize error?

---

6. Efficiency and Generalization

Dynamic Programming vs. Naive Calculation

A naive approach would compute the gradient for every weight independently. Backpropagation is an application of **Dynamic Programming**: it stores the intermediate "error" δ for each layer. Instead of recalculating the entire chain for every weight, it simply multiplies the current layer's local derivative by the already-computed error of the next layer. This transforms the complexity from exponential to linear.

Cross-Domain Link: This is identical to how a cache works in computer architecture. Instead of fetching data from the slow main memory (re-calculating the entire chain) every time, we store the most recent result in a fast register (the δ value) and reuse it for the next operation.

Computational Graphs

Modern frameworks (PyTorch, TensorFlow) treat the network as a **Directed Acyclic Graph (DAG)**. - **Forward Pass:** The graph records the sequence of operations. - **Backward Pass:** The framework

traverses the graph in reverse, applying the chain rule at each node. This allows for "Autograd," where you can change the network architecture without ever manually deriving a derivative again.

Real-World Application: Large Language Models (LLMs) In models like GPT-4, which contain billions of parameters, manual derivation of gradients is impossible. These models utilize "Automatic Differentiation" (the implementation of backpropagation on DAGs) to update weights across hundreds of Transformer layers. Specifically, the gradients are computed for the attention weights and feed-forward layers, allowing the model to learn complex linguistic patterns by propagating the error from the predicted next token all the way back to the input embeddings.